

EXP 2-Non-Preemptive Shortest Job First CPU Scheduling Algorithm

[Back To Practice](#)

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $\text{Turnaround Time (TAT)} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$
- $\text{Waiting Time (WT)} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$

Input Format

1. The first line of input contains an integer representing the number of processes

Select Language: C (GCC 9.2.0)

1 #include <stdio.h>
2
3 int main() {
4 int n, i, completed = 0, current_time = 0;
5
6 printf("Enter number of processes: ");
7 scanf("%d", &n);
8
9 int at[n], bt[n], ct[n], tat[n], wt[n], visited[n];
10
11 for(i = 0; i < n; i++) {
12 printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
13 scanf("%d %d", &at[i], &bt[i]);
14 visited[i] = 0;
15 }
16
17 while(completed < n) {
18 int min_index = -1;
19 int min_bt = 1e9;
20
21 // Find process with minimum burst time among arrived processes
22 for(i = 0; i < n; i++) {
23 if(at[i] <= current_time && visited[i] == 0) {
24 if(bt[i] < min_bt) {

Test Cases 2

Run Sample Cases

Run All Cases

[Back To Practice](#)

Q1

Save

Input Format

1. The first line of input contains an integer representing the number of processes.

2. The next lines contain two integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```

4
0 6
0 8
0 7
0 3

```

Select Language: C (GCC 9.2.0)

```

25         min_bt = bt[i];
26         min_index = i;
27     }
28 }
29 }
30
31 if(min_index == -1) {
32     // No process has arrived yet + move time forward
33     current_time++;
34 } else {
35     // Execute selected process
36     current_time += bt[min_index];
37     ct[min_index] = current_time;
38     visited[min_index] = 1;
39     completed++;
40 }
41 }
42
43 float total_tat = 0, total_wt = 0;
44
45 // Calculate TAT and WT
46 for(i = 0; i < n; i++) {
47     tat[i] = ct[i] - at[i];
48     wt[i] = tat[i] - bt[i];

```

Test Cases 2

Run Sample Cases

Run All Cases

```

49         total_tat += tat[i];
50         total_wt += wt[i];
51     }
52 }
53
54 printf("\n\nAverage Turnaround Time = %.2f", total_tat / n);
55 printf("\n\nAverage Waiting Time = %.2f\n", total_wt / n);
56
57 return 0;
58 }

```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input:

```

4
0 6
0 8
0 7
0 3

```

Expected Output:

```

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00

```

Output:

```

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

```